Chapter 00: Introduction to ENG3091

Khiem Nguyen – khiem.nguyen@glasgow.ac.uk

Contents

1 Introduction to C/C++ **1**
1.1 A small history . . . 1
1.1.1 Before C++, there was C: . . . 1
1.1.2 Then C was born: . . . 2
1.2 Introduction to C and C++'s philosophy . . . 2
1.3 Compiled vs Scripting Language (C++ vs Python) . . . 2
1.3.1 C++ (Compiled) . . . 2
1.3.2 Python (Scripting) . . . 2
1.4 What is C++ good at? . . . 2
1.5 Hello world in C++ . . . 2
1.6 Another program example . . . 3

2 Curriculum & Assessment **3**
2.1 Curriculum . . . 3
2.2 Curriculum: Extra when we have enough time . . . 4
2.3 What is **software engineering**? . . . 4
2.4 What is **software engineering**? . . . 4
2.5 The word **software engineering** is overstated in this course!!! . . . 4
2.6 Will learning Programming & Software Engineering be still relevant in AI era? . . . 4
2.7 Answer from Gemini . . . 5
2.8 Answer from CoPilot . . . 6
2.9 Answer from ChatGPT . . . 7
2.10 Assessment . . . 7
2.11 Performce of the students in the last few years . . . 8
2.12 How to learn this course . . . 8

1 Introduction to C/C++

1.1 A small history

1.1.1 Before C++, there was C:

- The C language was developed in 1972 by Dennis Ritchie at Bell Telephone lab, primarily as a systems programming language (to write operating systems).
- C was designed to give the programmer a lot of control, while allowing developers to write a program that could be run on different platforms.
- C ended up being so efficient and flexible that in 1973, Ritchie and Ken Thompson rewrote most of the Unix operating system using C.

1.1.2 Then C was born:

- C++ (pronounced "see plus plus") was developed by Bjarne Stroustrup at Bell Labs as an extension to C, starting in 1979.
- C++ adds many new features to the C language, and is perhaps best thought of as a superset of C.
- C++'s most notable innovation over C is that it supports object-oriented programming.

1.2 Introduction to C and C++'s philosophy

- The underlying design philosophy of C and C++ can be summed up as "trust the programmer" – which is both wonderful and dangerous.
- C++ is designed to allow the programmer a high degree of freedom to do what they want.
- this also means the language often won't stop you from doing things that don't make sense.
- C++ is a compiled language.

1.3 Compiled vs Scripting Language (C++ vs Python)

1.3.1 C++ (Compiled)

- **Performance:** Native machine code (Fast).
- **Errors:** Found at *Compile-time*.
- **Deployment:** Ship the binary (`a.out`).
- **Types:** Static (Strict).

1.3.2 Python (Scripting)

- **Performance:** Interpreted (Slower).
- **Errors:** Found at *Runtime*.
- **Deployment:** Ship the source code (`.py`).
- **Types:** Dynamic (Flexible).

1.4 What is C++ good at?

- Video games
- Real-time systems
- High-performance financial applications (e.g. high frequency trading)
- Graphical applications and simulations
- Productivity / office applications
- Embedded software
- Audio and video processing
- Artificial intelligence and neural networks

C++ also has a large number of high-quality 3rd party libraries available, which can shorten development times significantly.

1.5 Hello world in C++

- We need to compile this program to see the output. Watch me!

Listing 1 Hello ENG3091

```cpp
#include <iostream> // to use std::cout

int main() {    // Every program has function main()

    std::cout << "Welcome everybody to ENG3091!" << std::endl;  // note semicolor at the end
    std::cout << "\nWe shall learn:\n";                          // most of statement ends with sem
    std::cout << "(1) C++ Basics\n";

    std::cout << "(2) Object-oriented in C++\n";
    std::cout << "(3) Some standard libraries in C++\n";
    std::cout << "(4) Unified Modelling Language\n";

    return 0;           // The program returns "0", meaning it executes successfully.
}
```

1.6 Another program example

Listing 2 Do some maths

```cpp
#include <iostream> // to use std::cout

// We can do some mathematics.
int main() {
    double a = 5.5;
    double b = 4.5;
    std::cout << "a + b = " << a + b << std::endl;
    std::cout << "This program is almost useless\n";
    return 0;
}
```

} We need to compile this program to see the output. Watch me!

2 Curriculum & Assessment

2.1 Curriculum

1. **Basic C++**

   Variables, Functions, Control Flow

2. **Intermediate C++**

   References, Pointers, Template, Function Overriding, Dynamic Array

3. **Object-oriented Programming** (OOP)

   4 pillars of OOP: (1) Encapsulation, (2) Abstraction, (3) Inheritance, (4) Polymorphism

4. **Standard Template Library (STL)**

- Some commonly used STL containers: Vector, List, Array, ...
- Some commonly used STL algorithms: Sort, Find, Count, Unique, ...

5. **Unified Modelling Language**:

   In this course, we learn how to organize a large project by using class diagrams

2.2 Curriculum: Extra when we have enough time

6. **make** and **cmake** – Extra materials: Coverred if we have enough time.

   How to automate build process, how to install open-source libraries and how to build our own libraries

2.3 What is software engineering?

./figures/software_engineering_video.mp4

2.4 What is software engineering?

https://www.youtube.com/watch?v=4qSk8_AZwIg

2.5 The word software engineering is overstated in this course!!!

My opinions:

- You must be kidding me.
  - A software engineer's average salary varies greatly by location, experience, and company, but in the UK, it generally ranges from entry-level around $£25k - £35k$ up to $£70k+$ for senior roles, with London often paying more, while in the US, median total compensation can exceed $\$180k$, especially at top tech firms.
  - The School of Engineering asks me to teach Software Engineering in 10 weeks? Mission impossible!
- Can you become a software engineer after 10 weeks of learning C++ and class diagrams?
  - HELL NO
- Then, why do we even need to learn?
  - Whoever we want to become, we need to build the first brick.

2.6 Will learning Programming & Software Engineering be still relevant in AI era?

> *I am teaching C++ at School of Engineering, Uni of Glasgow. Many students believe that AI Chatbot like ChatGPT, Gemini and CoPilot will soon replace software engineering. What is your opinion about this?*

Let us ask Gemini, CoPilot and ChatGPT what they think.

2.7 Answer from Gemini

The short answer is: **AI is not replacing software engineering; it is automating the "syntax" while raising the stakes for "engineering."**

The consensus in early 2026 is that we are moving from a **Code-Centric** era to an **Orchestration-Centric** era. Here is a breakdown of why your students' future is safe, provided they evolve their definition of what a "software engineer" actually does.

---

1. The "70% Problem" and the Illusion of Completion

Students often see a chatbot spit out a 50-line C++ function and think, "My job is gone." However, in industry, writing the code is often only 20% of the work.

- **The Gap:** AI is excellent at "first drafts." But current models still struggle with the **last 30%**: debugging subtle race conditions, handling niche edge cases, and ensuring production-grade security.
- **The Risk:** AI makes "fast mistakes." Without a deep understanding of fundamentals (pointers, memory management, O-notation), a student becomes a "Vibe Coder"—someone who can't fix a system once the AI's "vibe" is wrong.

2. Shift in Roles: From "Author" to "Steward"

In 2026, the industry is seeing a fracture in the traditional role. We are seeing the rise of:

- **AI Orchestrators:** Engineers who don't just write code, but manage teams of "AI agents" to build complex systems.
- **Systems Architects:** Because AI can generate thousands of lines of code instantly, the

2.8 Answer from CoPilot

Not in the near future—and possibly never in the way people imagine. AI tools like ChatGPT, Gemini, and Copilot are **assistive technologies**, not autonomous engineers. They excel at:

- **Code generation for well-defined tasks**
- **Boilerplate reduction**
- **Debugging assistance**
- **Rapid prototyping**

But software engineering is much more than writing code. t involves:

- **Problem formulation**: Understanding real-world constraints and translating them into requirements.
- **System architecture**: Designing scalable, secure, and maintainable systems.
- **Domain-specific reasoning**: Applying physics, math, and engineering principles (especially in your field—thermal energy and multiscale materials).
- **Ethics, safety, and compliance**: AI cannot autonomously ensure these.

Why AI won't fully replace engineers

- **Contextual understanding**: AI lacks deep domain knowledge and judgment.
- **Creativity and innovation**: AI can remix patterns but struggles with truly novel solutions.
- **Responsibility**: Engineers are accountable for safety-critical systems; AI cannot take legal or ethical responsibility.

What will change?

- **Roles will evolve**: Engineers will spend less time on repetitive coding and more on design, validation, and integration.
- **AI literacy becomes essential**: Future engineers must know how to leverage AI effectively and responsibly.
- **Collaboration with AI**: Think of AI as a "junior assistant" that speeds up tasks but needs supervision

2.9 Answer from ChatGPT

From a professional and educational standpoint, the idea that AI tools such as ChatGPT, Gemini, and GitHub Copilot will *"replace"* software engineers in the near future is a significant overstatement. These tools will *transform* how software is developed — but not eliminate the need for human engineers. Below is a structured analysis you can share with your students.

---

1. Current Capabilities of AI Coding Tools

AI tools can already:

- Generate code snippets from natural-language prompts.
- Suggest completions within an IDE.
- Help with boilerplate code, refactoring, and documentation.
- Diagnose common errors and suggest fixes.

However:

- They lack deep semantic understanding of complex system requirements.
- They cannot make architectural decisions autonomously.
- They struggle with domain-specific knowledge without human guidance.

Summary: AI excels at *assisting* routine, well-scoped coding tasks, not independently solving complex engineering problems.

---

2. Why Software Engineering Isn't Merely "Coding"

Software engineering involves a broad set of skills and decisions that extend far beyond writing lines of code:

A. Requirements Engineering

Understanding stakeholder needs, negotiating priorities, and translating domain problems into technical

2.10 Assessment

- 2 Continuous Assessments: 25% + 25%
  - For each assessment: A group of 2 or 3 students will work on the same assignment and get the same grade (peer review grade is under consideration).
  - Please talk to each other and team up on your willing.
  - From my own experience, you tend to get better grades working in pair/group than working individually.
- 1 Paper Exam: 50%

2.11 Performce of the students in the last few years

Performance on the last three years

- Year 2022/2023 – Nick Bailey: 36 → On average, E2
- Year 2023/2024 – Nick Bailey: 35 → On average, E2
- Year 2024/2025 – Khiem Nguyen: 45 → On average, D2
- Mid-term assignment grades from me are also significantly higher than from Nick Bailey – **On average**: A3 (or probably A2) from me and A5 from Nick Bailey.

Question *Huge difference between mid-term assignments and final paper exam???*

Answer *Yes, ChatGPT can help in mid-term*, but not in the paper exam.

On average, I am confident to say I am generally more lenient than many other lecturers.

But, I cannot give points to a completely wrong answer.

2.12 How to learn this course

- As with any programming language, you need to:
  1. **practice**
  2. **practice**
  3. **practice**
- Can you use **AI Chatbot** to **learn** this course?
  - Hell, yes!
  - But you really need to "**learn**". There is a *hudge* difference between **"you understand"** and **"you think you understand"**.
- It is **foolish** to think that
  1. "I only need to worry about the big picture, AI Chatbot will do programming in the future."
  2. "AI Chatbot will replace software developers."
  3. "Why do we need to study programming in the era of AI Boom!"